

From Manual Server Checks to a Five-Minute Reporting Workflow

Infrastructure health reporting automation | Internal tooling | Individual initiative

At a glance

THE CHALLENGE		THE SOLUTION
Preparing infrastructure health reports required repeated server/database access, commands, and manual consolidation.		I independently built a shell-script workflow that collected metrics across systems and generated a consolidated, browser-readable HTML report.

BEFORE	AFTER	ADOPTION
~30–60 minutes per reporting run	~5 minutes per reporting run	Used regularly within a ~10-person team

My role and ownership

I identified the repetitive work, proposed the automation approach, designed and implemented the shell-script workflow, and produced the HTML reporting output. I used AI assistance for parts of the development. The team used the solution regularly during my employment.

Impact in one sentence

Estimated 83–92% reduction in report preparation time per run, based on approximately 30–60 minutes before versus five minutes after. These figures are recollected estimates, not independently audited measurements.

Case-study status: retrospective account of a real internal tool. Diagrams and example data in this document are reconstructed and sanitized; they are not screenshots of client infrastructure.

A recurring reporting bottleneck

The team prepared several types of operational reports roughly three to four times per day, and sometimes more frequently on request. To prepare them, a team member connected to Linux servers and databases, executed different commands or scripts, gathered outputs, and assembled the results for banking stakeholders, supervisors, and technical teams.

What the reports needed to communicate

- Current server resource usage, including CPU load and capacity-related metrics.
- Database health and status, including relevant operational checks.
- Monitoring and job/schedule status, plus backup recency and timing.
- A clear, time-stamped snapshot that non-operators could review in a browser.

Why existing tools were not enough

The database already retained diagnostic logs and alerts for incident investigation. The gap was not missing telemetry: it was the time needed to extract selected information from different systems and package it into a consistent, stakeholder-friendly snapshot.

Success criterion

Reduce manual preparation time while preserving the information needed for regular reporting. The tool was not intended to replace continuous monitoring, incident diagnosis, or database-native logging.

The original workflow

STEP	MANUAL PROCESS
1	Connect to the relevant Linux/database environments.
2	Run commands and scripts for different health checks.
3	Collect and reconcile outputs from multiple servers.
4	Format and share the reports with stakeholders.

One execution, one consolidated report

I implemented a shell-script-based workflow that could connect to the required environments, gather selected metrics, and write the results into a consolidated HTML report at a chosen server location. The report could then be opened in a browser and shared with the intended recipients.

Sanitized system flow

INPUTS	COLLECTION LAYER	OUTPUT
Linux servers; database environments; existing command/script outputs	On-demand shell script; multi-server connections; metric aggregation; report generation	Timestamped HTML health report; shared with team and stakeholders

Key product and technical choices

- **Reuse existing data:** Collect available operational metrics rather than rebuild monitoring and alert logging.
- **Automate the repetitive portion:** Replace multiple manual commands and output consolidation with a single script execution.
- **Make information accessible:** Use HTML for a structured, browser-readable snapshot rather than raw terminal output.
- **Keep execution on demand:** Support routine checkpoints and ad hoc reporting requests. Scheduling via crontab was possible but was not implemented.

Scope boundaries

Implemented: collection, consolidation, and HTML generation. Not implemented: unattended cron scheduling, automated distribution, a replacement monitoring platform, or a new real-time alerting system.

Operational considerations

In a production review, I would explicitly validate access permissions, command failures, partial-data handling, report timestamps, and safe distribution of reports. These are review considerations, not claims that every safeguard was implemented.

Observed outcome

The reporting workflow was adopted by the team during my employment. Preparation time was reduced from an estimated 30–60 minutes to about five minutes per run. I do not have verified information about whether the tool continued to be used after I left the organization.

Impact calculation — transparent assumptions

MEASURE	ESTIMATE
Time saved per run	25–55 minutes
Preparation-time reduction	~83–92%
If used 3 times/day	~75–165 minutes saved per day
If used 4 times/day	~100–220 minutes saved per day

Daily savings are illustrative extrapolations assuming comparable runs and frequency, not measured organization-wide outcomes.

What this project demonstrates for an APM role

- **Problem discovery:** Recognized a recurring operational friction point without being asked to build a new product.
- **Prioritization:** Focused on report generation rather than replacing existing diagnostic infrastructure.
- **Execution and ownership:** Built a usable internal tool independently, with limited AI assistance.
- **Adoption:** Delivered something teammates incorporated into recurring work.
- **Impact orientation:** Evaluated value using a before-and-after workflow metric.

What I would improve next

If extending the tool, I would assess demand for optional scheduling, stronger error handling, run-status indicators, report retention, and distribution controls. I would prioritize these against actual team needs rather than assume more automation is always better.

Interview-ready takeaway

“I noticed our team was spending 30–60 minutes repeatedly collecting and formatting health data that already existed in our systems. I built a shell-script workflow to consolidate the checks into a browser-readable HTML report. The team adopted it for regular reporting, bringing preparation down to

roughly five minutes per run. I deliberately focused on making reporting faster rather than replacing our existing monitoring tools.”